levelupedu.net | Coaching • Mentoring • Consulting

HANDS-ON ASSIGNMENT

PySpark RDD Session 02

Multi-Industry Practice Workbook

Data Engineering Masterclass

Course	Data Engineering Masterclass
Module	PySpark RDD — Session 02
Topics	groupByKey • mapValues • aggregateByKey • filter • distinct
Industries	Banking | Telecom | Healthcare | E-Commerce | Food Delivery | IT/IoT
Level	Beginner to Intermediate
Trainer	Sateesh Pabbathi | Microsoft MVP | levelupedu.net

"Learn by Doing — Real Industry. Real Data. Real PySpark."

About Your Trainer

Sateesh Pabbathi

Microsoft MVP

22+ Years Experience

Empowering 5,000+ Data Engineers Across India

Sateesh Pabbathi is a Microsoft MVP and founder of LevelUp Cloud Trainings. With 22+ years of industry experience, he has trained 5,000+ students in data engineering using real-world analogies drawn from companies like Zomato, Swiggy, HDFC Bank, and Flipkart.

Technologies Taught:

Microsoft Fabric • Databricks • ADF • PySpark • Python • SQL • GCP • AWS

- Interview-ready content — every concept tied to a real job question
- Hands-on labs — Google Colab, Databricks, Microsoft Fabric
- Story-driven lessons using Indian IT industry contexts
- Lifetime access to notes, assignments & lab guides

🌐 **levelupedu.net | YouTube • LinkedIn • Instagram • WhatsApp**

LevelUp
Coaching • Mentoring • Consulting

LevelUp
LevelUp

"Your success in data engineering is my mission. Let us level up — together."

— Sateesh Pabbathi

Assignment Instructions & Evaluation Criteria

How to Attempt

- Set up PySpark in Google Colab before starting. Session 01 setup commands are in your notes.
- Each exercise is independent — attempt in any order.
- Use ONLY the transformations specified in each task.
- Write your code in the answer boxes or a .py / Colab notebook file.
- Run your code and paste the actual output alongside your code.
- Add a 2–3 line comment explaining your logic for every task.

Sateesh Says: These exercises are modelled on real data engineering scenarios at HDFC Bank, Airtel, Apollo Hospitals, and Swiggy. When you solve these, you are not just doing homework — you are building your interview portfolio!

Evaluation Criteria

Criterion	Description	Marks
Correct Output	Code produces the expected output	**30**
Right Transformation	Correct use of the specified operation	**25**
Code Quality	Clean, readable, well-commented	**20**
Understanding	Brief explanation of your approach	**15**
Bonus Challenge	Optional advanced task	**10 (bonus)**

Quick Reference — Session 02 Operations

```
groupByKey()        => rdd.groupByKey().mapValues(list)
aggregateByKey()    => rdd.aggregateByKey((0,0), seqOp, combOp)
filter()            => rdd.filter(lambda x: x[1] > threshold)
distinct()          => rdd.distinct()
mapValues()         => rdd.mapValues(lambda v: transform(v))
```

LevelUp
LevelUp

Exercise 1 🏦 Banking & Finance — HDFC Bank

You are on HDFC Bank's analytics team. Use RDD operations to analyse customer transaction data for the monthly risk and revenue report.

Dataset: (customer_id, city, transaction_type, amount)

```
# HDFC Bank — Customer Transaction Log
# (customer_id, city, transaction_type, amount)
transactions = [
    ('C001','Mumbai','CREDIT',45000), ('C002','Delhi','DEBIT',12000),
    ('C001','Mumbai','DEBIT',8500),   ('C003','Bengaluru','CREDIT',75000),
    ('C002','Delhi','CREDIT',30000),  ('C004','Hyderabad','DEBIT',22000),
    ('C003','Bengaluru','DEBIT',18000),('C001','Mumbai','CREDIT',62000),
    ('C004','Hyderabad','CREDIT',95000),('C001','Mumbai','DEBIT',5000),
    ('C002','Delhi','CREDIT',40000),  ('C003','Bengaluru','CREDIT',30000)
]
txn_rdd = sc.parallelize(transactions)
```

Sateesh Says: Every NEFT transfer you do goes through a pipeline like this one. HDFC's fraud team runs filter() on millions of transactions every night looking for anomalies. You are building that system!

filter() Task 1.1 — Extract High-Value Transactions

Use filter() to extract only transactions where amount >= 50,000. These are flagged for RBI compliance reporting.

Input	`(customer_id, city, type, amount)`
Condition	`amount >= 50000`
Output	`All tuples where amount is 50,000 or more`

Starter Code:

```
high_value = txn_rdd.filter(lambda x: x[?] >= 50000)
high_value.collect()
```

Your Solution:

Actual Output:

LevelUp

groupByKey() Task 1.2 — List All Transactions per Customer

Use groupByKey() to group all transaction amounts for each customer. Use mapValues(list) to convert to a list. This gives the full transaction history per customer.

Key	customer_id
Value	list of all amounts for that customer
Use	groupByKey().mapValues(list)

Starter Code:

```
kv_rdd = txn_rdd.map(lambda x: (x[0], x[3]))
grouped = kv_rdd.groupByKey().mapValues(?)
grouped.collect()
```

Your Solution:

Actual Output:

aggregateByKey() Task 1.3 — Total + Count of Transactions per Customer

Use aggregateByKey() to compute both total_amount and transaction_count for each customer in a single pipeline.

zeroValue	(0, 0) — (sum, count)
seqOp	Add amount to sum, increment count by 1
Output	(customer_id, (total_amount, count))

Starter Code:

```
zero  = (0, 0)
seqOp = lambda acc, val: (acc[0]+val, acc[1]+1)
combOp= lambda a1, a2:  (a1[0]+a2[0], a1[1]+a2[1])
kv_rdd = txn_rdd.map(lambda x: (x[0], x[3]))
result = kv_rdd.aggregateByKey(zero, seqOp, combOp).collect()
```

LevelUp LevelUp

Your Solution:

Actual Output:

---

distinct() Task 1.4 — Find Unique Cities with Transactions

Use distinct() to get a list of unique cities that appear in the transaction data. No duplicates.

Input	`Extract city from each transaction`
Output	`Unique list of cities only`

Starter Code:

```
cities_rdd = txn_rdd.map(lambda x: x[1])
unique_cities = cities_rdd.distinct()
unique_cities.collect()
```

Your Solution:

Actual Output:

---

filter() + aggregateByKey() Task 1.5 — Average CREDIT Transaction per Customer

First filter() to keep only CREDIT transactions. Then use aggregateByKey() to calculate the average credit amount per customer.

LevelUp

Step 1	filter where transaction_type == 'CREDIT'
Step 2	aggregateByKey() to get (sum, count) per customer
Step 3	map() to compute average = sum / count

Starter Code:

```
credits = txn_rdd.filter(lambda x: x[?] == 'CREDIT')
kv = credits.map(lambda x: (x[0], x[3]))
agg = kv.aggregateByKey((0,0), lambda a,v:(a[0]+v,a[1]+1), lambda a1,a2:
(a1[0]+a2[0],a1[1]+a2[1]))
avg = agg.map(lambda x: (x[0], round(x[1][0]/x[1][1],2)))
avg.collect()
```

Your Solution:

Actual Output:

LevelUp
LevelUp

Exercise 2 📡 Telecom — Airtel CDR Analysis

Airtel's analytics team processes Call Detail Records to detect heavy users, identify popular call types, and build roaming reports for network planning.

Dataset: (caller_id, receiver_id, duration_mins, call_type, city)

```
# Airtel — Customer Call Detail Records (CDR)
# (caller_id, receiver_id, call_duration_mins, call_type, city)
cdr_data = [
    ('A001','A002',12,'Local','Hyderabad'), ('A002','A003',45,'STD','Delhi'),
    ('A001','A003',8, 'Local','Hyderabad'), ('A003','A001',30,'ISD','Mumbai'),
    ('A002','A001',15,'Local','Delhi'),     ('A004','A002',60,'STD','Chennai'),
    ('A001','A004',5, 'Local','Hyderabad'), ('A003','A004',25,'Local','Mumbai'),
    ('A002','A003',8, 'Local','Delhi'),     ('A004','A001',40,'ISD','Chennai'),
]
cdr_rdd = sc.parallelize(cdr_data)
```

Sateesh Says: Airtel processes 35 crore+ subscribers' CDRs daily. Every time you make a call, a record like this gets written to a Kafka topic and processed by pipelines exactly like this one!

filter() Task 2.1 — Extract Long Calls (duration > 20 mins)

Use filter() to identify calls longer than 20 minutes. These are premium usage customers for targeted data plan upselling.

Condition	`call_duration_mins > 20`
Output	`Tuples where duration exceeds 20 minutes`

Starter Code:

```
long_calls = cdr_rdd.filter(lambda x: x[?] > 20)
long_calls.collect()
```

Your Solution:

Actual Output:

LevelUp

distinct() Task 2.2 — Find Unique Call Types

Use distinct() to extract the unique types of calls in the dataset (Local, STD, ISD). Airtel's pricing team uses this for tariff segmentation.

Input	`Extract call_type from each record`
Output	`Unique call types only — no duplicates`

Starter Code:

```
types_rdd = cdr_rdd.map(lambda x: x[3])
unique_types = types_rdd.distinct()
unique_types.collect()
```

Your Solution:

Actual Output:

---

groupByKey() Task 2.3 — List All Receivers per Caller

Use groupByKey() to group all receivers called by each caller. This builds the 'contact list' view used for social graph analysis.

Key	`caller_id`
Value	`list of all receiver_ids called`

Starter Code:

```
kv = cdr_rdd.map(lambda x: (x[0], x[1]))
contacts = kv.groupByKey().mapValues(list)
contacts.collect()
```

Your Solution:

LevelUp
LevelUp

Actual Output:

---

aggregateByKey() Task 2.4 — Total + Average Call Duration per Caller

Use aggregateByKey() to compute total minutes and call count per caller. Then calculate the average call duration.

zeroValue	`(0, 0) — (total_mins, call_count)`
Output	`(caller_id, avg_duration) — round to 2 decimals`

Starter Code:

```
kv = cdr_rdd.map(lambda x: (x[0], x[2]))
agg = kv.aggregateByKey((0,0),
          lambda a,v:(a[0]+v,a[1]+1),
          lambda a1,a2:(a1[0]+a2[0],a1[1]+a2[1]))
avg = agg.map(lambda x: (x[0], round(x[1][0]/x[1][1],2)))
avg.collect()
```

Your Solution:

Actual Output:

---

filter() + groupByKey() Task 2.5 — ISD Callers and Their Contact List

Filter only ISD calls, then groupByKey to get the list of international contacts per caller.

Step 1	`filter() — keep only ISD calls`

Step 2	groupByKey().mapValues(list) — list of ISD receivers

Starter Code:

```
isd_calls = cdr_rdd.filter(lambda x: x[3] == ?)
kv = isd_calls.map(lambda x: (x[0], x[1]))
isd_contacts = kv.groupByKey().mapValues(list)
isd_contacts.collect()
```

Your Solution:

Actual Output:

LevelUp
LevelUp

Exercise 3 🏥 Healthcare — Apollo Hospitals

Apollo Hospitals' analytics team needs department-wise billing insights, high-cost patient identification, and average stay analysis for quarterly operations review.

Dataset: (patient_id, dept, diagnosis, days, bill, city)

```
# Apollo Hospitals — Patient Records
# (patient_id, department, diagnosis, days_admitted, bill_amount, city)
patients = [
    ('P001','Cardiology','Hypertension',3,45000,'Hyderabad'),
    ('P002','Orthopaedics','Fracture',5,62000,'Mumbai'),
    ('P003','Cardiology','Heart Failure',8,120000,'Delhi'),
    ('P004','Neurology','Migraine',2,18000,'Bengaluru'),
    ('P005','Orthopaedics','Knee Replacement',6,95000,'Hyderabad'),
    ('P006','Gastroenterology','Gastritis',1,9500,'Chennai'),
    ('P007','Neurology','Stroke',9,135000,'Mumbai'),
    ('P008','Cardiology','Arrhythmia',4,55000,'Delhi'),
    ('P009','Gastroenterology','IBS',2,12000,'Bengaluru'),
    ('P010','Orthopaedics','Shoulder Surgery',4,78000,'Hyderabad')
]
patients_rdd = sc.parallelize(patients)
```

Sateesh Says: Healthcare data engineering is life-saving. Faster billing analytics means quicker insurance approvals. The aggregateByKey() you write today could eventually speed up a patient's discharge!

filter() Task 3.1 — Identify High-Bill Patients (>= 75,000)

Use filter() to flag patients with bills >= Rs 75,000. These require pre-authorisation from insurance companies.

Condition	`bill_amount >= 75000`
Output	`Patient records with high bills`

Starter Code:

```
high_bill = patients_rdd.filter(lambda x: x[?] >= 75000)
high_bill.collect()
```

Your Solution:

Actual Output:

groupByKey() Task 3.2 — All Diagnoses per Department

Use groupByKey() to collect all diagnoses within each department. Helps the medical committee track which conditions are most common per specialty.

Key	`department`
Value	`list of all diagnoses in that department`

Starter Code:

```
kv = patients_rdd.map(lambda x: (x[1], x[2]))
dept_diagnoses = kv.groupByKey().mapValues(?)
dept_diagnoses.collect()
```

Your Solution:

Actual Output:

aggregateByKey() Task 3.3 — Average Bill and Stay per Department

Use aggregateByKey() to compute total bill, total days, and patient count per department. Then calculate average bill and average stay.

zeroValue	`(0, 0, 0) — (sum_bill, sum_days, count)`
Output	`(dept, avg_bill, avg_days) — round to 2 decimals`

Starter Code:

```
kv = patients_rdd.map(lambda x: (x[1], (x[4], x[3])))
# zeroValue=(0,0,0) for (sum_bill, sum_days, count)
zero  = (0, 0, 0)
seqOp = lambda acc,v: (acc[0]+v[0], acc[1]+v[1], acc[2]+1)
combOp= lambda a1,a2: (a1[0]+a2[0], a1[1]+a2[1], a1[2]+a2[2])
agg   = kv.aggregateByKey(zero,seqOp,combOp)
```

LevelUp

```
result= agg.map(lambda x: (x[0], round(x[1][0]/x[1][2],2), round(x[1][1]/x[1]
[2],2)))
result.collect()
```

Your Solution:

Actual Output:

---

distinct() Task 3.4 — Find Unique Cities with Patients

Use distinct() to find all unique cities where Apollo has treated patients in this dataset.

Input	`Extract city from each patient record`
Output	`List of unique cities — no duplicates`

Starter Code:

```
city_rdd = patients_rdd.map(lambda x: x[5])
unique_cities = city_rdd.distinct()
unique_cities.collect()
```

Your Solution:

Actual Output:

LevelUp

filter() + aggregateByKey() Task 3.5 — Avg Bill for Long-Stay Patients (> 4 days)

Filter patients admitted more than 4 days, then compute average bill per department for these long-stay patients.

Step 1	`filter() — days_admitted > 4`
Step 2	`aggregateByKey() — total bill and count per dept`
Step 3	`map() — compute average`

Starter Code:

```
long_stay = patients_rdd.filter(lambda x: x[?] > 4)
kv = long_stay.map(lambda x: (x[1], x[4]))
agg = kv.aggregateByKey((0,0),lambda a,v:(a[0]+v,a[1]+1),lambda a1,a2:
(a1[0]+a2[0],a1[1]+a2[1]))
avg = agg.map(lambda x: (x[0], round(x[1][0]/x[1][1],2)))
avg.collect()
```

Your Solution:

Actual Output:

LevelUp

Exercise 4 🍔 Food Delivery — Swiggy

Swiggy's data team analyses order patterns to power cuisine recommendations, detect repeat orders, and compute customer loyalty scores.

Dataset: (order_id, customer_id, restaurant, cuisine, items, value, city)

```
# Swiggy — Order Records
# (order_id, customer_id, restaurant, cuisine, items, order_value, city)
orders = [
    ('O001','U101','Paradise Biryani','Hyderabadi',2,480,'Hyderabad'),
    ('O002','U102','Mainland China','Chinese',3,960,'Mumbai'),
    ('O003','U101','Dominos Pizza','Italian',1,399,'Hyderabad'),
    ('O004','U103','Saravana Bhavan','South Indian',4,640,'Chennai'),
    ('O005','U104','Burger King','American',2,320,'Delhi'),
    ('O006','U102','Paradise Biryani','Hyderabadi',3,720,'Mumbai'),
    ('O007','U105','McDonald India','American',4,580,'Bengaluru'),
    ('O008','U103','Pizza Hut','Italian',2,550,'Chennai'),
    ('O009','U104','Haldirams','North Indian',3,430,'Delhi'),
    ('O010','U101','Paradise Biryani','Hyderabadi',2,480,'Hyderabad'),
]
orders_rdd = sc.parallelize(orders)
```

Sateesh Says: That Paradise Biryani recommendation you see on Swiggy every Friday evening? That is powered by groupByKey() on your order history. U101 ordered from Paradise Biryani twice — that's a repeat customer signal!

distinct() Task 4.1 — Find Unique Restaurants Ordered From

Use distinct() to find all unique restaurants in the dataset. Swiggy's catalog team monitors this to track active restaurant partners.

Input	`Extract restaurant name from each order`
Output	`Unique restaurant names — no duplicates`

Starter Code:

```
rest_rdd = orders_rdd.map(lambda x: x[2])
unique_rest = rest_rdd.distinct()
unique_rest.collect()
```

Your Solution:

Actual Output:

groupByKey() Task 4.2 — Order History per Customer

Use groupByKey() to collect all order_ids for each customer. This builds the customer order history used for loyalty tier assignment.

Key	`customer_id`
Value	`list of all order_ids placed by that customer`

Starter Code:

```
kv = orders_rdd.map(lambda x: (x[1], x[0]))
history = kv.groupByKey().mapValues(list)
history.collect()
```

Your Solution:

Actual Output:

filter() + distinct() Task 4.3 — Customers Who Ordered Hyderabadi Cuisine

First filter() to keep only Hyderabadi cuisine orders. Then extract unique customer IDs using distinct().

Step 1	`filter() — cuisine == 'Hyderabadi'`
Step 2	`map() to get customer_id`
Step 3	`distinct() to get unique customers`

Starter Code:

```
hyd_orders = orders_rdd.filter(lambda x: x[3] == 'Hyderabadi')
customers = hyd_orders.map(lambda x: x[1]).distinct()
customers.collect()
```

LevelUp

Your Solution:

Actual Output:

---

aggregateByKey() Task 4.4 — Total Spend + Order Count per Customer

Use aggregateByKey() to compute total spend and number of orders for each customer. Then derive average order value.

zeroValue	`(0, 0) — (total_value, order_count)`
Output	`(customer_id, total_spend, avg_order_value)`

Starter Code:

```
kv = orders_rdd.map(lambda x: (x[1], x[5]))
agg = kv.aggregateByKey((0,0),
          lambda a,v:(a[0]+v,a[1]+1),
          lambda a1,a2:(a1[0]+a2[0],a1[1]+a2[1]))
result = agg.map(lambda x: (x[0], x[1][0], round(x[1][0]/x[1][1],2)))
result.collect()
```

Your Solution:

Actual Output:

LevelUp

filter() + groupByKey() Task 4.5 — Cuisines Ordered in Each City

Filter orders where order_value > 500, then groupByKey() to list all cuisine types available per city.

Step 1	`filter() — order_value > 500`
Step 2	`map() to (city, cuisine)`
Step 3	`groupByKey().mapValues(list)`

Starter Code:

```
premium = orders_rdd.filter(lambda x: x[5] > 500)
kv = premium.map(lambda x: (x[6], x[3]))
city_cuisines = kv.groupByKey().mapValues(list)
city_cuisines.collect()
```

Your Solution:

Actual Output:

LevelUp LevelUp

BONUS CHALLENGE — IoT / Smart Factory *Tata Motors Production Line*

Combine filter + distinct + groupByKey + aggregateByKey + sortBy in one pipeline!

Dataset: Tata Motors Assembly Line Sensor Readings

```
# (machine_id, shift, sensor_reading, alert_type, line)
factory_data = [
    ('M001','Morning',88,'NORMAL','LineA'),  ('M002','Morning',96,'HIGH','LineB'),
    ('M001','Evening',72,'NORMAL','LineA'),
('M002','Evening',108,'CRITICAL','LineB'),
    ('M003','Morning',65,'NORMAL','LineC'),  ('M001','Night',94,'HIGH','LineA'),
    ('M002','Night',101,'CRITICAL','LineB'),
('M003','Evening',88,'NORMAL','LineC'),
    ('M004','Morning',55,'NORMAL','LineA'),  ('M004','Evening',99,'HIGH','LineA'),
    ('M003','Night',91,'HIGH','LineC'),
('M001','Morning',88,'NORMAL','LineA'),
]
factory_rdd = sc.parallelize(factory_data)
```

Your tasks (in order):

1. filter() — Keep only readings >= 90 (abnormal readings only)
2. distinct() — Remove duplicate (machine_id, shift) entries if any
3. groupByKey() — Group all abnormal readings per machine (machine_id as key)
4. aggregateByKey() — Compute total abnormal reading sum and count per machine
5. map() — Calculate average abnormal reading per machine
6. sortBy() — Sort machines by average abnormal reading (highest first)
7. EXTRA: find the distinct alert_types per production line using groupByKey

Your Solution:

Expected Output:

Sateesh Says: If you solved the bonus, you just built Tata Motors' predictive maintenance alert system. Put this in your resume: 'Built PySpark RDD IoT sensor pipeline for anomaly detection and machine health scoring'. TCS and Wipro interviewers love this!

LevelUp
LevelUp

Quick Reference — Session 02 Cheat Sheet

Operation	Syntax	When to Use
groupByKey()	`rdd.groupByKey().mapValues(list)`	Need full list of values per key
aggregateByKey()	`rdd.aggregateByKey((0,0),seqOp,combOp)`	Need sum+count or multiple aggregates
filter()	`rdd.filter(lambda x: x[1]>90)`	Keep only matching records
distinct()	`rdd.distinct()`	Remove duplicate elements
mapValues()	`rdd.mapValues(lambda v: list(v))`	Transform value only, keep key